ANÁLISIS DEL RETO

Nicolas Liz Figueroa, 202611971, n.liz@uniandes.edu.co

Juan Pablo Martinez, 202611361, jp.martinezr1@uniandes.edu.co

Alvaro Andres Villate, 202616607, a.villatel@uniandes.edu.co

Requerimiento 1

Descripción

Durante la carga de datos cada venta se inserta en un árbol rojo-negro (RBT) propio de su marca. Los árboles se guardan en una tabla de hash (linear probing) cuya llave es el nombre de la marca. La llave de cada árbol es la tupla (discount_percent, order_date, order_id): el recorrido en orden entrega las órdenes por descuento ascendente y, en empate, por fecha ascendente, y el order_id hace que la llave sea única para que ninguna venta se sobrescriba.

Para resolver el requerimiento se busca el árbol de la marca en la tabla de hash, se pide con rbt.values el rango de llaves que va desde (desc_min, fecha mínima) hasta (desc_max, fecha máxima) y se recorre ese resultado para armar la lista de respuesta y acumular el costo total final.

Entrada	Catálogo con las estructuras de datos, nombre de la marca, descuento mínimo y descuento máximo (en porcentaje, límites incluidos).
Salidas	Tiempo de ejecución (ms), número total de órdenes que cumplen el filtro, costo total final (USD) y la lista de órdenes ordenada por descuento ascendente y, en empate, por fecha ascendente. La vista imprime las primeras 5 y las últimas 5 si hay más de 10.
Implementado (Sí/No)	Sí. Implementado por Nicolas Liz Figueroa (requerimiento individual).

Análisis de complejidad

Pasos	**Complejidad**
Buscar el árbol de la marca en la tabla de hash (mlp.get)	O(1) promedio
Obtener del árbol las órdenes con descuento en el rango (rbt.values)	O(log m + k)
Recorrer las k órdenes: agregarlas a la lista de respuesta y sumar el costo final	O(k)
TOTAL	***O(log n + k)***

Análisis

El requerimiento no recorre las n ventas del catálogo. El índice por marca se construye una sola vez durante la carga de datos (un rbt.put de O(log n) por venta, O(n log n) en total), de modo que cada consulta solo paga la búsqueda en la tabla de hash, el descenso por el árbol (m es el número de órdenes de la marca, m ≤ n) y el recorrido de los k resultados.

En el peor caso, cuando el rango de descuentos abarca todas las órdenes de la marca, k es proporcional a n y el costo es O(n). En los demás casos es mucho menor que recorrer toda la lista, y como el RBT mantiene su altura en O(log n) la búsqueda del rango es eficiente incluso con el archivo large.

Requerimiento 2

Descripción

Durante la carga de datos cada venta se inserta en un RBT propio de su país, y los árboles se guardan en una tabla de hash (linear probing) cuya llave es el país. La llave de cada árbol es la tupla (customer_rating, order_date, order_id), por lo que el recorrido en orden entrega calificación ascendente y, en empate, fecha ascendente.

Para resolver el requerimiento se busca el árbol del país, se obtiene con rbt.values el rango [calif_min, calif_max] y se recorre ese resultado insertando cada orden al inicio de una lista enlazada (add_first). Así el orden se invierte y la respuesta queda de mayor a menor calificación y, en empate, de la fecha más reciente a la más antigua. Si hay más de 10 órdenes se toman con sub_list las primeras 5 y las últimas 5.

Entrada	Catálogo con las estructuras de datos, país, calificación mínima y calificación máxima (límites incluidos).
Salidas	Tiempo de ejecución (ms), número total de órdenes que cumplen el filtro, costo total final (USD) y las listas primeros_5 y ultimos_5 (si hay 10 órdenes o menos, todas quedan en primeros_5 y ultimos_5 queda vacía). La vista muestra order_id, fecha, marca, modelo, país, canal, calificación, precio final e ingresos.
Implementado (Sí/No)	Sí. Implementado por Alvaro Andres Villate (requerimiento individual).

Análisis de complejidad

Pasos	**Complejidad**
Buscar el árbol del país en la tabla de hash (mlp.get)	O(1) promedio
Obtener del árbol las órdenes con calificación en el rango (rbt.values)	O(log m + k)
Recorrer las k órdenes: add_first en la lista enlazada (invierte el orden) y sumar el costo final	O(k)
Obtener las primeras 5 y las últimas 5 órdenes (sub_list)	O(k)
TOTAL	***O(log n + k)***

Análisis

Igual que en el requerimiento 1, el índice por país se construye una sola vez al cargar los datos (O(n log n)), así que cada consulta no recorre las n ventas: depende de la altura del árbol del país (m es el número de órdenes del país, m ≤ n) y del número de resultados k. Insertar con add_first cuesta O(1) por orden, por eso invertir el orden no agrega más que O(k).

Con el archivo small, la consulta para USA con calificaciones entre 3.5 y 5.0 (967 órdenes) tomó 3.31 ms. Con un archivo más grande la misma consulta (3839 órdenes) tomó 7.71 ms. El tiempo crece con el número de resultados k y no con el tamaño total del archivo, lo que coincide con el análisis. En el peor caso, con k proporcional a n, el costo es O(n).

Requerimiento 3

Descripción

Se recorre la lista de ventas del catálogo y, por cada venta, se verifica que el canal de venta, la categoría y el rango de precio final cumplan el filtro. Las ventas que lo cumplen se acumulan (conteo y costo total) y se insertan en un RBT con llave (final_price_usd, order_date, order_id), de modo que el árbol las deja ordenadas por precio final ascendente y, en empate, por fecha ascendente.

Luego se obtiene con value_set la lista ordenada del recorrido en orden del árbol. Si tiene más de 10 elementos se toman con sub_list las primeras 5 y las últimas 5; de lo contrario se retorna completa.

Entrada	Catálogo con las estructuras de datos, canal de venta, categoría del producto, precio final mínimo y precio final máximo (límites incluidos).
Salidas	Tiempo de ejecución (ms), número total de órdenes que cumplen el filtro, costo total final (USD) y las listas primeros_5 y ultimos_5, ordenadas por precio final ascendente y, en empate, por fecha ascendente.
Implementado (Sí/No)	Sí. Implementado por Juan Pablo Martinez (requerimiento individual).

Análisis de complejidad

Pasos	**Complejidad**
Recorrer la lista de ventas y evaluar el filtro (canal, categoría y rango de precio)	O(n)
Insertar en el RBT las k ventas que cumplen el filtro (rbt.put)	O(k log k)
Obtener la lista ordenada con el recorrido en orden del árbol (value_set)	O(k)
Obtener las primeras 5 y las últimas 5 órdenes (sub_list)	O(k)
TOTAL	***O(n + k log k) ≤ O(n log n)***

Análisis

A diferencia de los requerimientos 1 y 2, este requerimiento no usa un índice precalculado: en cada consulta se recorre toda la lista de ventas (O(n)). El RBT se usa para ordenar las k ventas que cumplen el filtro en O(k log k), sin necesidad de un algoritmo de ordenamiento adicional.

Como k ≤ n, en el peor caso (todas las ventas cumplen el filtro) el costo total es O(n log n). Una posible mejora sería indexar las ventas durante la carga por (canal, categoría), como se hizo con la marca en el requerimiento 1.

Requerimiento 4

Descripción

Se recorre la lista de ventas y, por cada venta dentro del rango de fechas y con el nivel de ingresos del cliente indicado, se actualiza en una tabla de hash (llave: la marca) un diccionario con el conteo de órdenes, la suma de final_price_usd, la suma de revenue_usd y la orden con mayor revenue_usd (en empate, la más reciente por fecha).

Después cada marca se inserta en una cola de prioridad (min-heap) con prioridad (-total, nombre): sale primero la marca con más órdenes y, en empate, la que va antes en el alfabeto. Se extraen las N primeras marcas y, al sacarlas de la cola, se calculan su precio final promedio y su ingreso promedio.

Entrada	Catálogo con las estructuras de datos, rango de fechas [fecha inicial, fecha final] (AAAA-MM-DD, límites incluidos), la cantidad N de marcas a mostrar y el nivel de ingresos del cliente (Low, Medium o High).
Salidas	Tiempo de ejecución (ms), número total de marcas consideradas y, para cada marca: nombre, número de órdenes que cumplen el filtro, precio final promedio, ingreso generado promedio y la orden con mayor ingreso (order_id, fecha, marca y modelo, país, canal, precio final e ingreso). Las marcas van ordenadas por número de órdenes descendente y, en empate, por nombre.
Implementado (Sí/No)	Sí. Implementado por Juan Pablo Martinez (requerimiento grupal).

Análisis de complejidad

Pasos	**Complejidad**
Recorrer las n ventas, filtrar por fechas y nivel de ingresos y acumular por marca en la tabla de hash (mlp.get / mlp.put)	O(n)
Recorrer las b marcas e insertarlas en la cola de prioridad (pq.insert)	O(b log b)
Extraer las N marcas con más órdenes (pq.remove) y calcular sus promedios	O(N log b)
TOTAL	***O(n + b log b)***

Análisis

Aquí b es el número de marcas (b ≤ n) y N ≤ b. El término dominante es el recorrido de la lista de ventas, O(n), porque las operaciones de la tabla de hash son O(1) en promedio y la comparación de fechas se hace sobre cadenas con formato AAAA-MM-DD.

La cola de prioridad solo maneja las marcas, que son muy pocas (6 en este dataset), así que su costo es despreciable frente al recorrido de las ventas. En la práctica el requerimiento crece de forma lineal con el número de ventas, igual que los requerimientos 5 y 6, que siguen el mismo esquema de agrupar en una tabla de hash y ordenar con una cola de prioridad.

Requerimiento 5

Descripción

Se recorre la lista de ventas y, por cada venta del país y del rango de fechas indicados, se actualiza en una tabla de hash (llave: la marca) un diccionario con el conteo de órdenes, la suma de customer_rating, la suma de revenue_usd y la orden con más unidades vendidas (en empate, la de mayor order_id).

Después se calcula el promedio de calificación de cada marca y se inserta la marca en una cola de prioridad (min-heap) con prioridad (-promedio, nombre): el promedio más alto sale primero y, en empate, sale primero el nombre que va antes en el alfabeto. Finalmente se extraen de la cola las N mejores marcas.

Entrada	Catálogo con las estructuras de datos, país de venta, rango de fechas [fecha inicial, fecha final] (AAAA-MM-DD, límites incluidos) y la cantidad N de marcas a mostrar.
Salidas	Tiempo de ejecución (ms), número total de marcas consideradas y, para cada marca: nombre, número de órdenes, calificación promedio, ingreso generado promedio y la orden con más unidades vendidas (order_id, fecha, marca y modelo, país y canal, unidades y calificación). Las marcas van ordenadas por calificación promedio descendente y, en empate, por nombre ascendente.
Implementado (Sí/No)	Sí. Implementado por Alvaro Andres Villate (requerimiento grupal).

Análisis de complejidad

Pasos	**Complejidad**
Recorrer las n ventas, filtrar por país y rango de fechas y acumular por marca en la tabla de hash (mlp.get / mlp.put)	O(n)
Recorrer las b marcas, calcular el promedio e insertarlas en la cola de prioridad (pq.insert)	O(b log b)
Extraer las N mejores marcas de la cola (pq.remove)	O(N log b)
TOTAL	***O(n + b log b)***

Análisis

Aquí b es el número de marcas (b ≤ n) y N ≤ b. El término dominante es el recorrido de la lista de ventas, O(n), porque las operaciones de la tabla de hash son O(1) en promedio. El costo de la cola de prioridad depende solo del número de marcas, que es muy pequeño (6 en este dataset), así que en la práctica el requerimiento se comporta de forma lineal en el número de ventas.

Con el archivo small, la consulta para Germany entre 2021-01-01 y 2023-12-31 con N = 5 tomó 6.12 ms. Con un archivo más grande tomó 42.47 ms, un crecimiento aproximadamente lineal con el número de ventas, como indica el análisis.

Requerimiento 6

Descripción

Se recorre la lista de ventas y, por cada venta dentro del rango de fechas y del rango de precios base, se agrega a la lista de ventas de su marca en una tabla de hash (llave: la marca) y se acumula la suma de descuentos.

Luego, por cada marca se calcula el promedio de descuento, la desviación estándar poblacional de discount_percent (un recorrido de sus ventas) y la orden con el descuento más cercano al promedio (otro recorrido; en empate, la de menor order_id). La marca se inserta en una cola de prioridad (min-heap) con prioridad (desviación, promedio, nombre), de modo que salen primero las marcas más consistentes y, en empate, la de menor promedio de descuento. Finalmente se extraen las N primeras.

Entrada	Catálogo con las estructuras de datos, rango de fechas [fecha inicial, fecha final] (AAAA-MM-DD), rango de precios base en USD [precio mínimo, precio máximo] (límites incluidos) y la cantidad N de marcas a mostrar.
Salidas	Tiempo de ejecución (ms), número total de marcas resultado del filtro y, para cada una de las N marcas más consistentes: nombre, número de órdenes analizadas, promedio del descuento, desviación estándar del descuento (puntos porcentuales) y la orden con el descuento más cercano al promedio (order_id, fecha, marca y modelo, país y canal, descuento).
Implementado (Sí/No)	Sí. Implementado por Alvaro Andres Villate y Nicolas Liz Figueroa (requerimiento grupal).

Análisis de complejidad

Pasos	**Complejidad**
Recorrer las n ventas, filtrar por fechas y precio base y agruparlas por marca en la tabla de hash (mlp.get / mlp.put, al.add_last)	O(n)
Por cada marca: promedio, desviación estándar y orden más cercana al promedio (dos recorridos de sus k_i ventas, con la suma de los k_i ≤ n)	O(n)

Insertar las b marcas en la cola de prioridad (pq.insert)	O(b log b)
Extraer las N marcas más consistentes (pq.remove)	O(N log b)
TOTAL	***O(n + b log b)***

Análisis

Aquí b es el número de marcas (b ≤ n) y N ≤ b. Como cada venta pertenece a una sola marca, los recorridos por marca suman en total a lo sumo n ventas, así que calcular la desviación estándar y la orden más cercana al promedio de todas las marcas sigue siendo O(n). La cola de prioridad solo maneja las b marcas, por lo que su costo es despreciable frente al recorrido de las ventas.

Con el archivo small, la consulta con fechas entre 2021-01-01 y 2023-12-31, precios base entre 50 y 200 y N = 5 tomó 16.84 ms. Con un archivo más grande tomó 87.38 ms, un crecimiento aproximadamente lineal con el número de ventas que concuerda con el análisis.